To use AR on web

Concept need for AR on Web

- Three.js is a powerful, open-source JavaScript library that simplifies the process of creating and displaying 3D graphics and animations in web browsers using WebGL. https://threejs.org/
-
- "A-Frame" refers to a web framework, written in JavaScript, that simplifies the creation of 3D and WebXR scenes, including AR experiences, using a HTML.
https://aframe.io/docs/1.7.0/introduction/
-
- <a-scene> is the main container where all 3D objects are placed. It initializes the A-Frame environment.
-
- <a-entity position="0 1.6 4">: Creates an entity to hold the camera. The position attribute sets the initial position of the camera (0 meters on the X-axis, 1.6 meters on the Y-axis (eye level), and 4 meters back on the Z-axis).
- <a-camera></a-camera>: This tag defines the camera in the scene, through which the user views the VR environment. Since it's nested within the a-entity, it inherits the entity's position.
- **Directional Light** simulates a light source (like the sun) coming from (x=1, y=1, z=1).
- **Ambient Light** provides a general soft lighting across the scene.
- **cube** at (x=0, y=1, z=-3), rotated **45° around the Y-axis**.
- **Animation:** Rotates from 0 45 0 to 0 90 0 **every second (dur: 1000ms)** and loops infinitely. **As code.**

A-Frame lifecycle methods

- init()

Called once when the component is attached to an entity; used for initial setup, variables, and event listeners.

- update(oldData)

Called after init() and whenever component data changes; used to react to property or state updates.

- play()

Called when the component or scene starts or resumes; used to activate behaviors.

- tick(time, timeDelta)

Called on every frame; used for animations, movement, or continuous updates.

- pause()

Called when the scene or component is paused; used to temporarily stop actions.

- remove()

Called when the component or entity is removed; used for cleanup like removing listeners or objects.

Steps for server (optional)

1: Check node, npm and express is installed

```
node -v
npm -v
npm list express
```

2: Create server.js by code

```
const express = require('express');
const path = require('path');
const app = express();
// Serve static files from the "public" directory
app.use(express.static(path.join(__dirname, 'public')));
// Start the server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
console.log(`Server is running on http://localhost:${PORT}`);
});
```

Exercise 1:

Displaying GLB Model on image Target

Steps 1: Download GLB Model inside folder public

Step 2: create .mind file

https://hiukim.github.io/mind-ar-js-doc/tools/compile/

Step 3: create File say Index.html in folder public

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
  <meta name="mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-capable" content="yes">
  <title>MindAR GLB Viewer</title>
```

```
  <script src="https://aframe.io/releases/1.5.0/aframe.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/mind-ar@1.2.4/dist/mindar-image-
aframe.prod.js"></script>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    html, body { overflow: hidden; width: 100vw; height: 100vh; }
  </style>
</head>
<body>
  <a-scene
    mindar-image="imageTargetSrc: ./new.mind; autoStart: true; uiLoading: yes; uiError: yes;
uiScanning: yes;"
    color-space="sRGB"
    renderer="colorManagement: true; physicallyCorrectLights: true; antialias: false; precision:
medium"
    vr-mode-ui="enabled: false"
    device-orientation-permission-ui="enabled: false"
    loading-screen="enabled: false">

    <a-assets timeout="30000">
      <a-asset-item id="model1" src="./model.glb"></a-asset-item>
    </a-assets>

    <!-- IMPORTANT: correct camera for mobile -->
    <a-camera position="0 0 0" look-controls="false" near="0.01" far="1000"></a-camera>

    <a-entity mindar-image-target="targetIndex: 0">
      <a-entity
        gltf-model="#model1"
        position="0 0 0.1"
        rotation="90 0 0"
        scale="0.4 0.4 0.4"
        animation-mixer>
      </a-entity>
    </a-entity>

  </a-scene>
</body>
</html>
```

Code explanation

autoStart = camera opens automatically

timeout="30000" gives **30 seconds** to load — important on slow mobile networks.

far="1000" = renders objects very far away means if distance between camera and image more than 1000 no render

mindar-image="imageTargetSrc: ./new.mind; autoStart: true; uiLoading: yes; uiError: yes; uiScanning: yes;"

Property	Meaning
imageTargetSrc	location of compiled image target file
autoStart:true	AR starts automatically
uiLoading	show loading screen
uiError	show error messages
uiScanning	show scanning UI

vr-mode-ui="enabled: false"

Remove VR Buttorn

Camera: look control false: disable mouse rotation

animation-mixer : enabled all animation if exist in 3D model

Flow
Camera
|
▼
MindAR Image Tracking
|
▼
Detect Target Image
|
▼
A-Frame Scene
|
▼
Load GLB Model
|
▼
Display 3D Object

//imp

Asset Vs Entity

<a-assets> is used to **preload resources** before the scene starts. : its like keep in cube

<a-entity> is used to **create objects in the scene**. : its like cube act like scale,rotate etc

//if camera is attached in laptop run code by simple server or use Server,js which is outside the public folder

Why server:

To avoid **CORS** Error

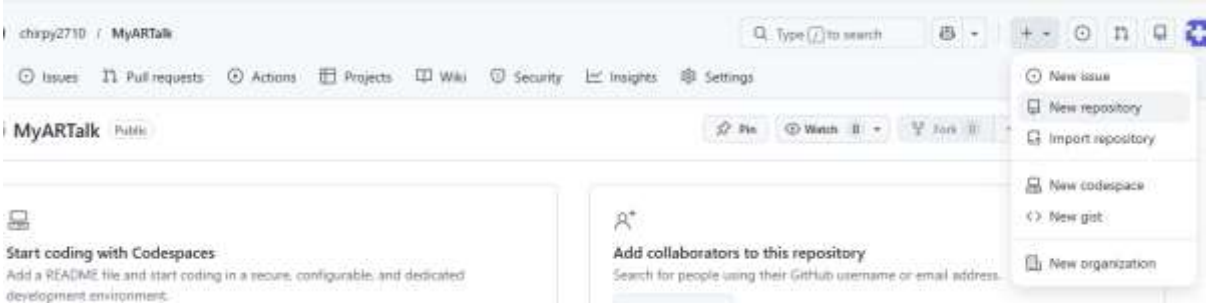
CORS = Cross-Origin Resource Sharing

- It is a **browser security rule**.
- It prevents a webpage from loading files from another domain unless that domain allows it.

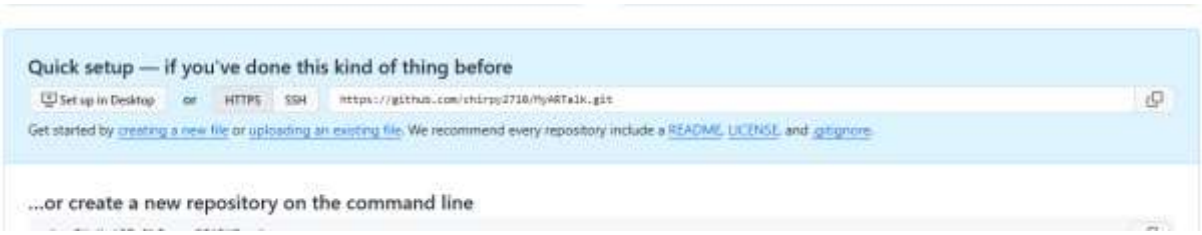
Or use github

1. open https://github.com/

2. create new repository by click +

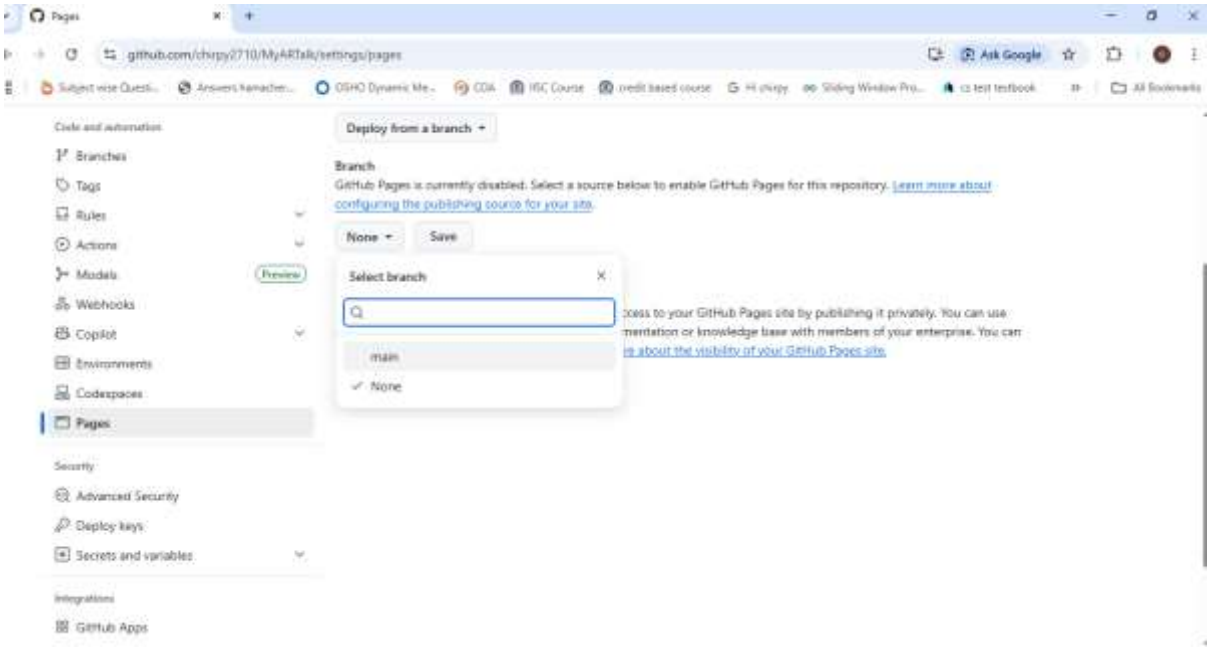


3. click create files or upload files

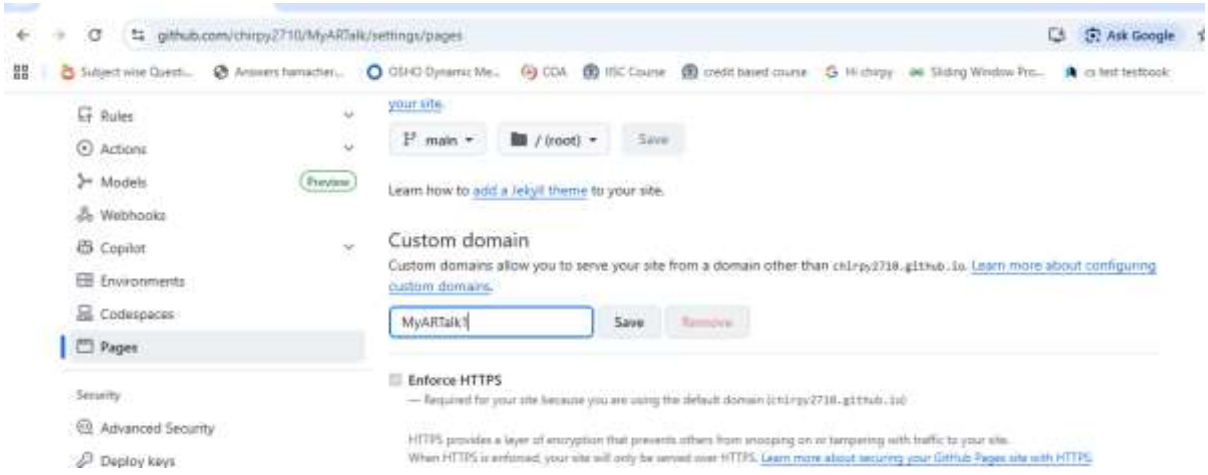


4. upload file model.glb & .Mind file

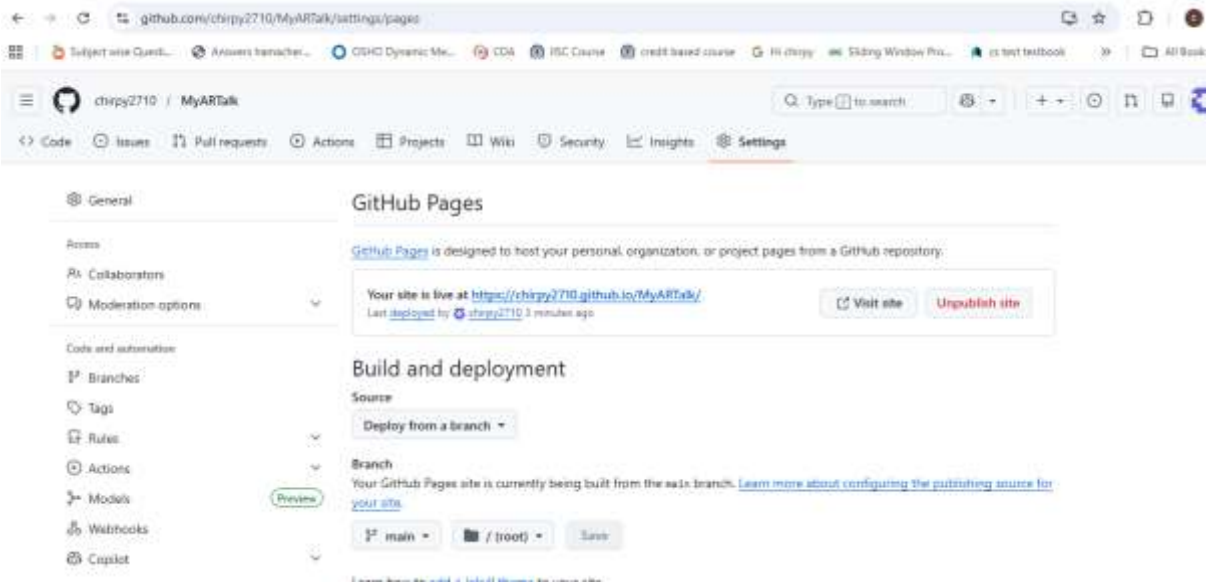
5. Setting -> pages -> set property as below click save



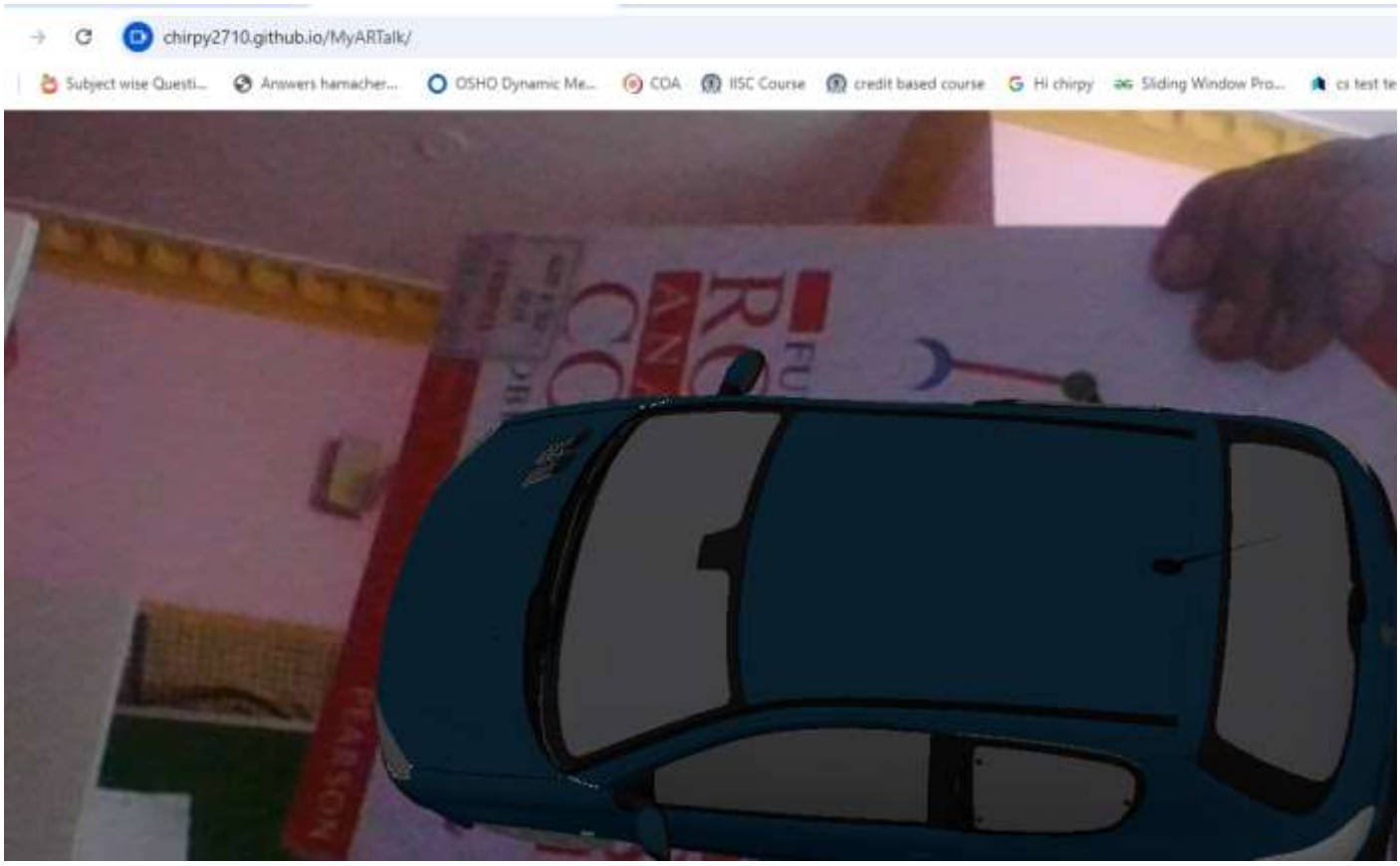
6. give name & Click Save



7. Open link


github.com/chirpy2710/MyARTalk/settings/pages
chirpy2710 / MyARTalk
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
General
Collaborators
Moderation options
Branches
Tags
Rules
Actions
Models
Preview
Webhooks
Copilot
GitHub Pages
GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.
Your site is live at https://chirpy2710.github.io/MyARTalk/
Visit site
Unpublish site
Build and deployment
Source
Deploy from a branch
Branch
main
/ (root)
Save

Output:


chirpy2710.github.io/MyARTalk/
Subject wise Questi...
Answers hamacher...
OSHO Dynamic Me...
COA
IISC Course
credit based course
Hi chirpy
Sliding Window Pro...

Exercise 2: Use WEB XR

Step 1: Create index.html

```
<html>
  <head>
    <script src="https://aframe.io/releases/1.2.0/aframe.min.js"></script>
    <script src="./components.js"></script>
    <style>
      #overlay {
        position: fixed;
        top: 100px;
        padding: 10px;
        z-index: 1001;
      }
    </style>
  </head>
  <body>
```

```
    <a-scene webxr="requiredFeatures:hit-test; optionalFeatures:dom-overlay;
referenceSpaceType:local; overlayElement:#overlay" background="transparent: true">

      <!-- Replace a-box with gltf-model entity -->
      <a-entity
        id="box"
        gltf-model="https://raw.githubusercontent.com/chirpy2710/ARwebnew/main/model.glb"
        scale="0.1 0.1 0.1"
        visible="false">
      </a-entity>

      <a-entity ar-reticle geometry="primitive: ring; radiusInner: 0.15; radiusOuter: 0.2" rotation="-
90 0 0" visible="false"></a-entity>
    </a-scene>
    <div id="overlay">

    </div>
  </body>
</html>
```

Step 2: create component.js

```
AFRAME.registerComponent('ar-reticle', {

  init: function () {

    this.xrHitTestSource = null;
    this.viewerSpace = null;
    this.refSpace = null;

    const sceneEl = this.el.sceneEl;
    const reticle = this.el;
    const model = document.getElementById('box');

    sceneEl.renderer.xr.addEventListener('sessionstart', async () => {

      const session = sceneEl.renderer.xr.getSession();

      // Viewer space
      this.viewerSpace = await session.requestReferenceSpace('viewer');

      // Hit test source
      this.xrHitTestSource = await session.requestHitTestSource({
        space: this.viewerSpace
      });

      // Local reference space
      this.refSpace = await session.requestReferenceSpace('local');

      console.log("AR Started");
```

```
    // Place model on tap
    session.addEventListener('select', () => {

      if (!reticle.getAttribute('visible')) return;

      const position = reticle.object3D.position;

      model.object3D.position.copy(position);
      model.setAttribute('visible', true);

      console.log("Model placed");
    });

  });

  sceneEl.renderer.xr.addEventListener('sessionend', () => {

    this.xrHitTestSource = null;
    this.viewerSpace = null;
    this.refSpace = null;

    console.log("AR Ended");
  });
},

tick: function () {

  const sceneEl = this.el.sceneEl;

  if (!sceneEl.is('ar-mode')) return;
  if (!this.xrHitTestSource || !this.refSpace) return;

  const frame = sceneEl.frame;
  if (!frame) return;

  const hitTestResults = frame.getHitTestResults(this.xrHitTestSource);

  if (hitTestResults.length > 0) {

    const pose = hitTestResults[0].getPose(this.refSpace);

    const matrix = new THREE.Matrix4();
    matrix.fromArray(pose.transform.matrix);

    const position = new THREE.Vector3();
    position.setFromMatrixPosition(matrix);

    this.el.object3D.position.copy(position);
    this.el.setAttribute('visible', true);

  } else {
```

```
    this.el.setAttribute('visible', false);
  }
 }
});
```

//code explaination

Concept	Code Example	Purpose	Explanation
Custom Component	AFRAME.registerComponent('ar-reticle', {})	Create reusable AR behavior	Registers a new component called **ar-reticle** that can be attached to an entity.
Initialization Function	init: function () {}	Runs once when component loads	Used to initialize variables and set up event listeners.
Tick Function	tick: function () {}	Runs every frame	Called **~60 times per second** to update AR tracking and reticle position.
Scene Reference	this.el.sceneEl	Access the A-Frame scene	Allows interaction with the scene renderer and XR session.
Renderer XR	sceneEl.renderer.xr	Access WebXR engine	Used to control AR session and access camera tracking data.
XR Session Start	sessionstart event	Detect when AR begins	Triggered when the AR session starts on the device.
XR Session End	sessionend event	Detect when AR stops	Used to reset AR variables when session ends.
XR Session Object	getSession()	Access AR session	Gives control over AR input, hit testing, and reference spaces.
Reference Space	requestReferenceSpace('viewer')	Define coordinate system	Determines how the device position is tracked in AR space.
Viewer Space	'viewer'	Device camera reference	Represents the position of the phone camera in AR.
Local Reference Space	'local'	World tracking	Provides a stable world coordinate system for placing objects.

Concept	Code Example	Purpose	Explanation
Hit Test Source	requestHitTestSource()	Surface detection	Enables detection of real-world surfaces like tables or floors.
Reticle Entity	const reticle = this.el	AR pointer	Represents the visual marker showing detected surface.
Model Reference	document.getElementById('box')	Access 3D model	Retrieves the model that will be placed in AR.
User Tap Event	session.addEventListener('select')	Detect user interaction	Triggered when the user taps the screen or presses AR controller.
Model Placement	model.object3D.position.copy(position)	Place object	Moves the model to the reticle position.
Visibility Control	setAttribute('visible', true)	Show or hide objects	Used to display the reticle or model when needed.
Frame Data	sceneEl.frame	AR frame data	Contains camera tracking and hit-test results for current frame.
Hit Test Results	frame.getHitTestResults()	Surface detection results	Returns detected surfaces from AR tracking.
Pose	getPose()	Surface position/orientation	Provides transformation data of detected surface.
Matrix Conversion	THREE.Matrix4()	Transform matrix	Converts WebXR pose data into usable 3D coordinates.
Position Extraction	setFromMatrixPosition()	Get coordinates	Extracts x,y,z position from transformation matrix.
Reticle Position Update	this.el.object3D.position.copy(position)	Move reticle	Updates reticle to detected surface location.
Visibility Logic	if (hitTestResults.length > 0)	Surface detection check	Shows reticle only when a surface is detected.

Flow

Camera Scan

↓

WebXR Hit Testing

↓

Surface Detected

↓

Reticle Position Updated

↓

User Tap

↓

3D Model Placed

Step 2: Use same glb model using link

Step 3:use chrome canary if ur mobile chrome not support webxr still not working install ARcore

Output: